Islamic University of Technology

Relational Database Management System Lab

CSE 4508

LibraTrack

Project Report

TAHSIN BIN REZA
210041106

RAFIUL AREFEEN ISLAM
210041114

ADIB AHMED
210041122

April 30, 2025

Contents

1 Overview 2
1.1 Motivation . . . 2
1.2 Stakeholders . . . 2
1.3 Features . . . 2
1.3.1 Member . . . 3
1.3.2 Admin/Librarian . . . 4

2 Tech Stack 7
2.1 JavaFX . . . 7
2.2 Maven . . . 7
2.3 MySQL . . . 7
2.4 IntelliJ IDEA . . . 7

3 RDBMS Design 7
3.1 Tables . . . 7
3.2 Normalization . . . 8
3.2.1 First Normal Form (1NF) . . . 8
3.2.2 Second Normal Form (2NF) . . . 9
3.2.3 Third Normal Form (3NF) . . . 9
3.3 Entity-Relationship Diagram . . . 9

4 SQL Features Used 9
4.1 General Queries . . . 9
4.2 Procedures . . . 10
4.2.1 CalculatePenalty() . . . 10
4.2.2 UpdateBookStatus() . . . 10
4.3 Triggers . . . 11

5 Challenges 11

6 Conclusion 11

1 Overview

LibraTrack is a simple Library Management System made with JavaFX and MySQL. It offers functionality to borrow books from library as well as automatic penalty tracking for overdue books. It also simplifies managing the book inventory.

1.1 Motivation

The motivations behind this project were:

1. To simplify library operations and reduce paperwork.
2. To offer a student-friendly interface for book borrowing and returns.
3. To provide an automated penalty calculation system based on overdue books.
4. To enhance accountability with user-based sessions and roles.

This project also showcases the capabilities of a relational database management system very nicely. This was a key factor in choosing this project.

1.2 Stakeholders

There are 3 stakeholders in LibraTrack:

- Members: Members can request books, return them, and monitor overdue penalties.
- Admin/Librarian: Librarians manage the entire book lifecycle including users, books, and requests.
- SysAdmins: Maintains and enhances the system.

1.3 Features

The features differ based on who is using the system (Members or Admins). A login screen is presented first.

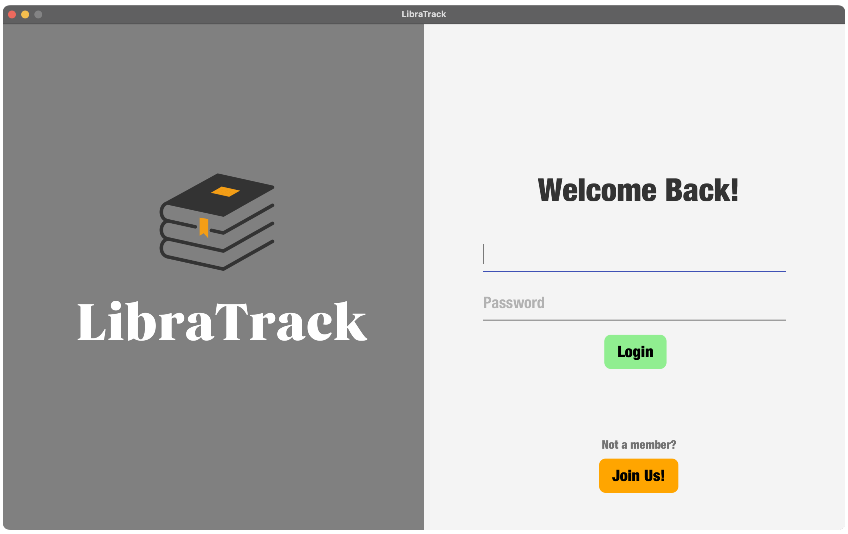

Figure 1: Login Page

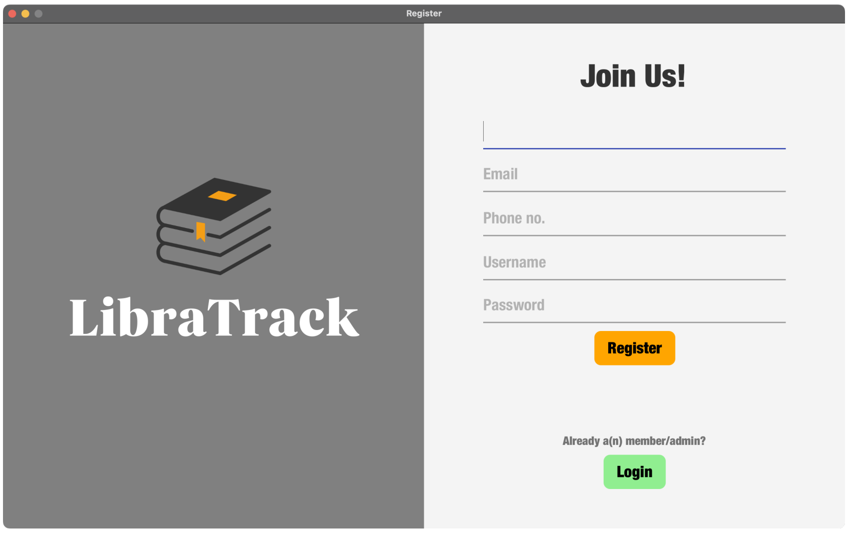

Figure 2: Register Page

The system automatically determines whether a user is a member or an admin based on the value of isAdmin column. Someone can also register as a new member from the register page.

1.3.1 Member

The dashboard for the member shows the count of books issued and the total amount of penalty to be paid (if there is any).
Members can do the following:

1. Select a book from the list of available books and request it to be issued to them.
2. Select a book from the already issued books and return it to the library.

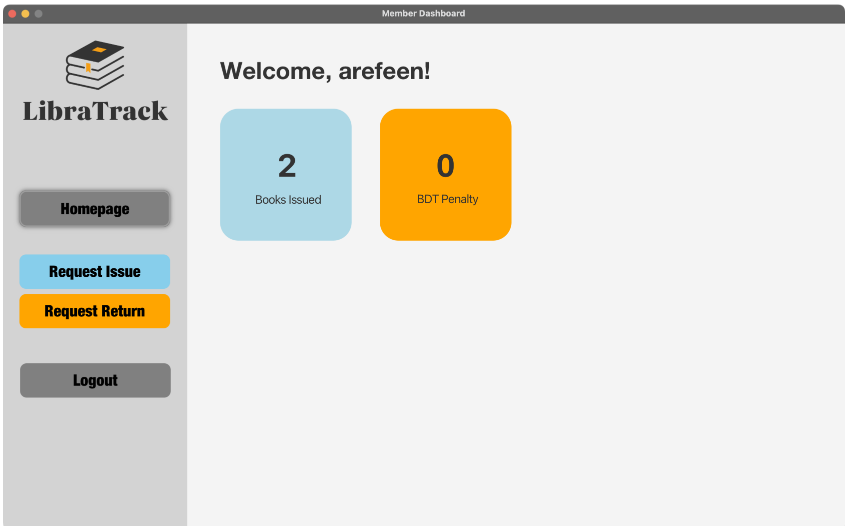

Figure 3: Member Dashboard

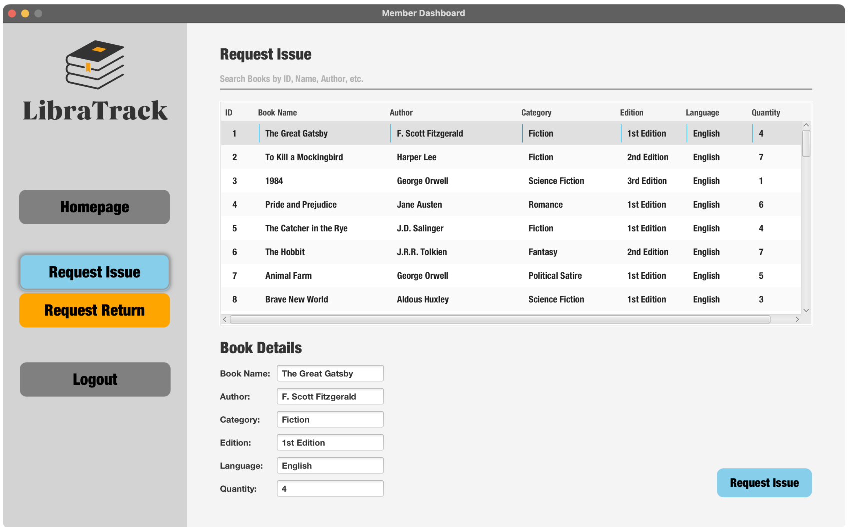

Figure 4: Request a book to be issued

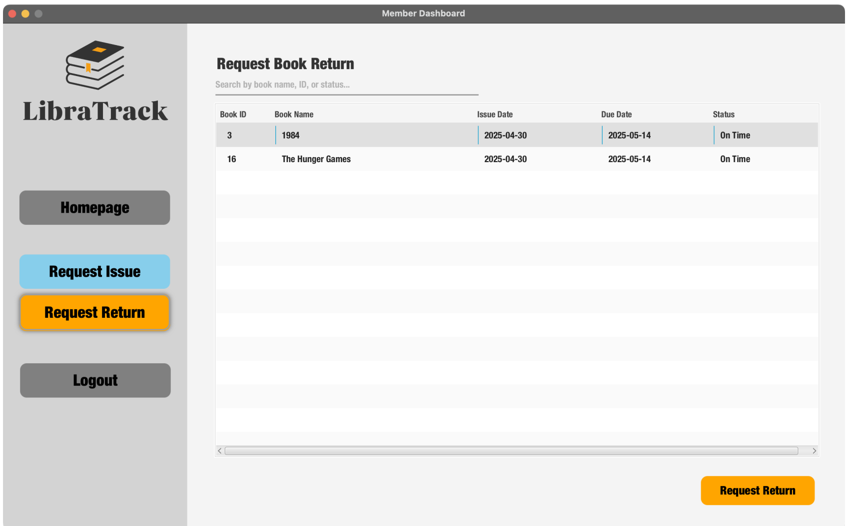

Figure 5: Return a book

1.3.2 Admin/Librarian

The dashboard for the admin shows the following things: count of issued books, count of users with penalty, number of pending requests (issue and return) and total number of books in the library.
Admins can do the following:

1. Edit member info and add or delete members.
2. Edit existing book details, add new books and delete existing books.
3. Issue any book to any user. Particularly useful in learning scenarios where some people taking some course are bound to require a particular book, so it can be issued without them even asking for it.
4. Accept issues requests from members.

5. Accept return requests from users. The penalty is also calculated and shown at this time if there is any.

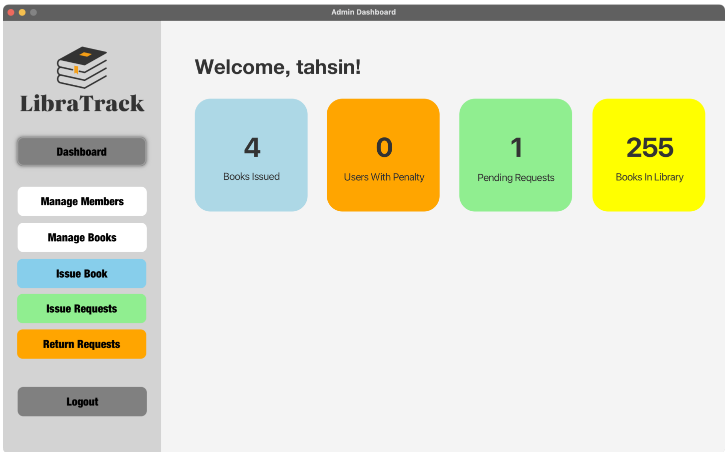

Figure 6: Admin dashboard

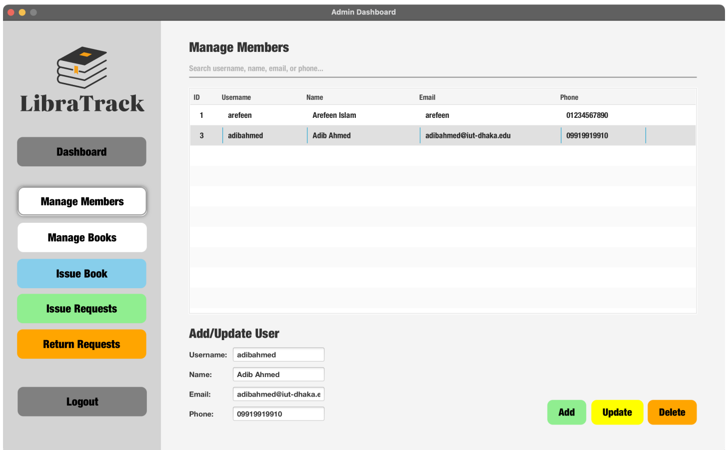

Figure 7: Manage members page

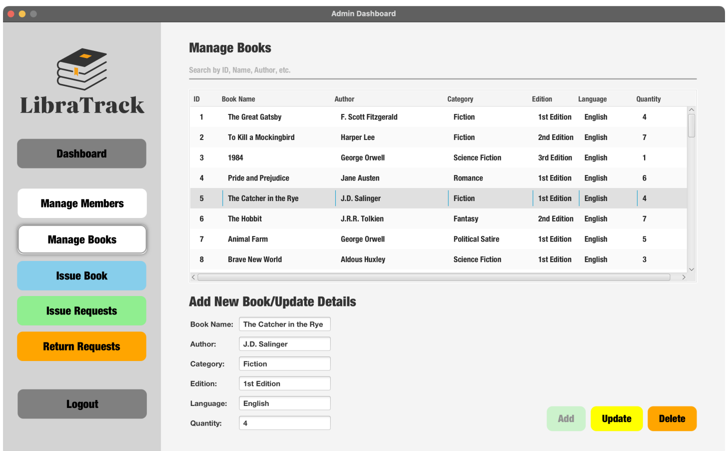

Figure 8: Manage Books Page

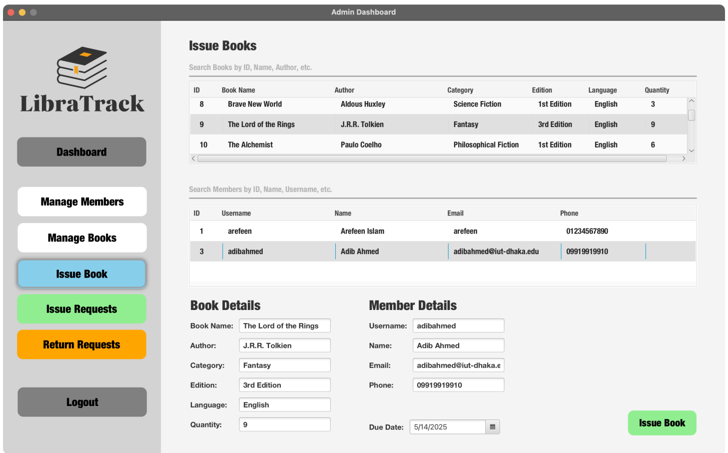

Figure 9: Issue Book Page

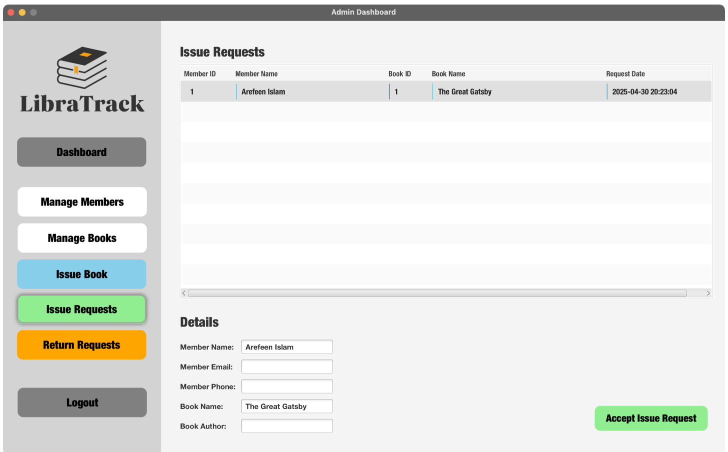

Figure 10: Manage Issue Requests Page

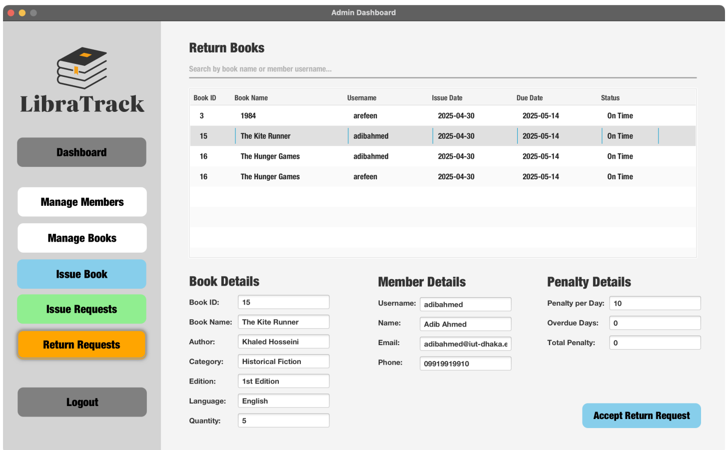

Figure 11: Manage Return Requests Page

2 Tech Stack

2.1 JavaFX

JavaFX was used to build the GUI for the system. It was chosen as it provides a modern and interactive user interface with features like scene builders, styling with CSS, and support for event-driven programming.

2.2 Maven

Maven was chosen as the build automation and dependency management tool. It simplified project setup, ensured consistent builds, and allowed easy integration of external libraries and frameworks.

2.3 MySQL

MySQL serves as the backend database for storing and managing user data, books, and transactions. It was chosen because of its reliability, support for features like procedures, triggers and cursors and easy integration with Java through JDBC.

2.4 IntelliJ IDEA

IntelliJ IDEA was our IDE of choice. It was chosen because of its rich featureset: intelligent suggestions, support for JavaFX and Maven and seamless integration with version control systems.

Additionally Github was used for version control.

3 RDBMS Design

This section focuses on the design of the database that is used to keep track of books and users.

3.1 Tables

Here are the tables and their columns. Primary key is indicated with an asterisk.

1. **books**
    - book_id *
    - book_name
    - author_name
    - category
    - edition
    - language
    - quantity

2. **issued_book_details**
    - id *
    - book_id
    - member_id
    - issue_date
    - due_date
    - status
3. **request_issue**
    - id *
    - member_id
    - book_id
    - request_date
4. **request_return**
    - id *
    - member_id
    - issue_id
    - request_date
5. **users**
    - user_id *
    - username
    - password_hash
    - name
    - email
    - phone_no
    - isAdmin
    - created_at

3.2 Normalization

The database follows the third normal form (3NF). This ensures there is no redundancy or problematic dependency within the data.

3.2.1 First Normal Form (1NF)

All of the fields in all of the tables contain atomic data. No column contains multiple values. For example in the `books` table, the `book_name` field contains only the title of the book and nothing else.

3.2.2 Second Normal Form (2NF)

All non-key attributes are dependent on a primary key. For example in the `users` table, all of the details are fully dependent on `user_id` primary key.

3.2.3 Third Normal Form (3NF)

No transitive dependency exists in any table.

3.3 Entity-Relationship Diagram

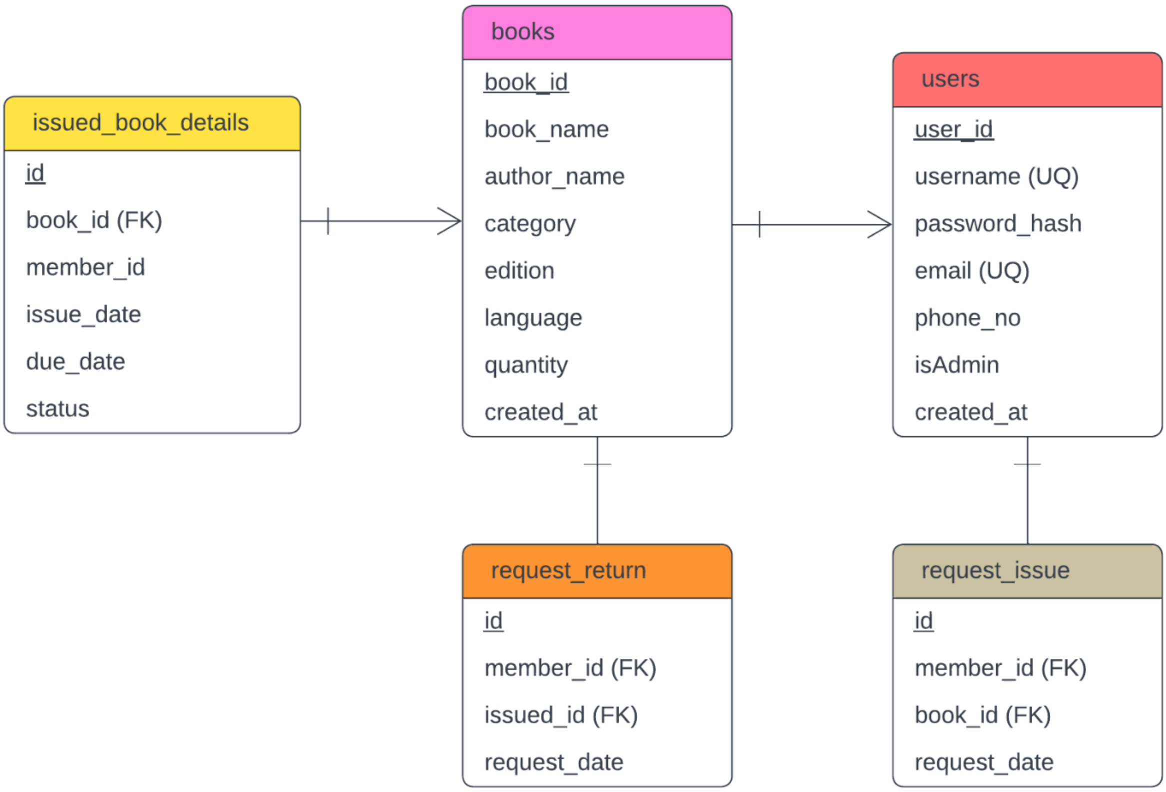

Figure 12: ER Diagram for LibraTrack's Database

4 SQL Features Used

This section focuses on the methods used to interact with the database.

4.1 General Queries

A lot of general queries were used to perform CRUD operations on the database. Below are some examples.

Retrieving hashed password and admin status for a particular user:

```sql
SELECT password_hash, isAdmin
FROM users
WHERE username = ?
```

Finding total number of requests for admin dashboard:

```sql
SELECT
    (SELECT COUNT(*) FROM request_issue) +
    (SELECT COUNT(*) FROM request_return)
    AS totalRequests
```

Finding number of users with a penalty:

```sql
SELECT COUNT(DISTINCT member_id) AS usersWithPenalty
FROM issued_book_details
WHERE DATEDIFF(CURDATE(), due_date) > 0
```

Issuing a book to a user:

```sql
INSERT INTO issued_book_details (book_id, member_id, issue_date,
   due_date, status)
VALUES (?, ?, CURRENT_DATE, ?, 'On Time')
```

Updating book details:

```sql
UPDATE books
SET book_name = ?,
    author_name = ?,
    category = ?,
    edition = ?,
    language = ?,
    quantity = ?
WHERE book_id = ?
```

4.2 Procedures

2 procedures are used: one for calculating the penalty based on the difference between current date and due date and another for updating book status to either on time or overdue.

4.2.1 CalculatePenalty()

```sql
CREATE DEFINER=`root`@`localhost` PROCEDURE `CalculatePenalty`()
BEGIN
    UPDATE issued_book_details
    SET
        status = CASE
                    WHEN CURDATE() > due_date THEN 'Overdue'
                    ELSE 'On Time'
                 END,
        overdue_days = GREATEST(DATEDIFF(CURDATE(), due_date), 0),
        penalty_amount = GREATEST(DATEDIFF(CURDATE(), due_date), 0) *
            penalty_per_day;
END
```

4.2.2 UpdateBookStatus()

```sql
CREATE DEFINER=`root`@`localhost` PROCEDURE `UpdateBookStatus`()
BEGIN
    -- Update status based on current date and due date
    UPDATE issued_book_details
```

```sql
    SET status = CASE
        WHEN due_date < CURDATE() THEN 'Overdue'
        ELSE 'On Time'
    END;
END
```

4.3 Triggers

One trigger was used called decrement_book_quantity that automatically that when attempting to issue a book checks whether it is available and if it is, then decrements the count of the said book from the inventory. Otherwise it signals with an error code.

```sql
CREATE DEFINER=`root`@`localhost` TRIGGER `decrement_book_quantity` BEFORE INSERT ON `issued_book_details` FOR EACH ROW BEGIN
    DECLARE current_quantity INT;

    -- Fetch current quantity of the book
    SELECT quantity INTO current_quantity
    FROM books
    WHERE book_id = NEW.book_id;

    -- Check if the book is available
    IF current_quantity <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Book is not available for issuing';
    END IF;

    -- Decrement the book quantity
    UPDATE books
    SET quantity = quantity - 1
    WHERE book_id = NEW.book_id;
END
```

5 Challenges

Some challenges we faced are as follows:

1. Syncing database status updates with GUI in real time.
2. Handling user concurrency in issue/return scenarios.
3. Designing an intuitive interface for both admin and member views.
4. Ensuring data consistency across request and issued book tables.

6 Conclusion

LibraTrack enhances traditional library management by offering a digital, user-friendly system for both users and librarians. With its JavaFX GUI, MySQL backend and automatic penalties and status tracking, it provides a robust platform for streamlined library operations.

Github Repository